

# Module 2

## Syntax Analysis

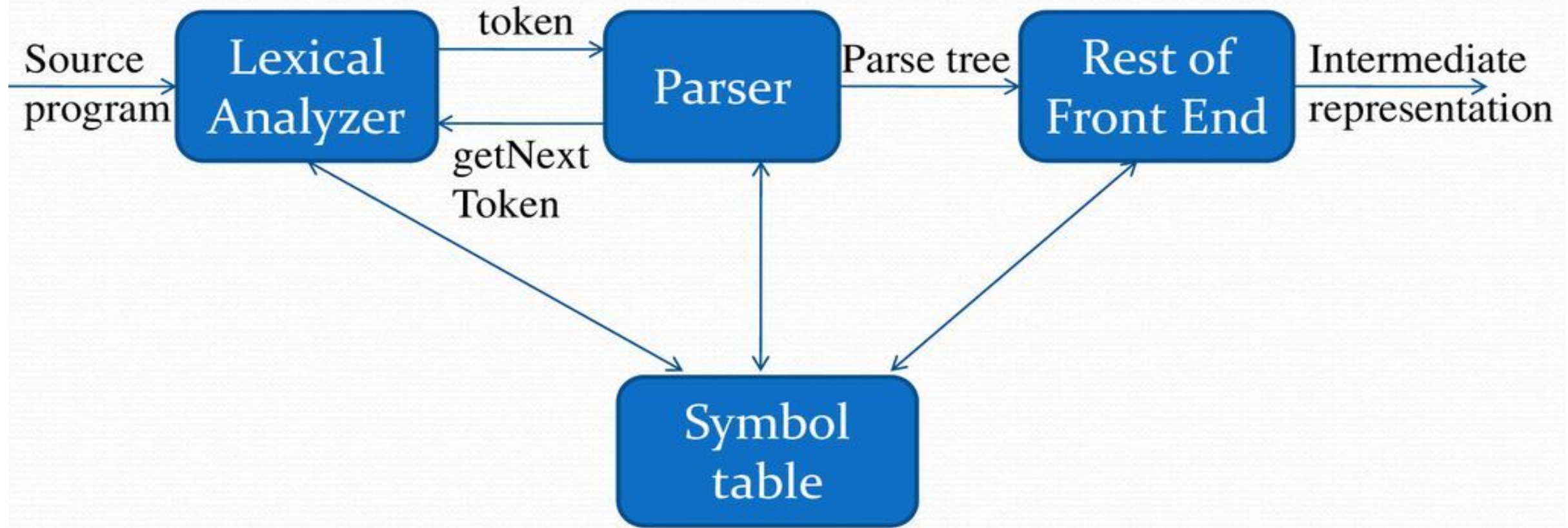
# Syllabus:

- Introduction
- Context Free Grammars
- Writing a Grammar
- Top-down Parsers
- Bottom-up Parsers

# The Role of the Parser:

- **Definition:** **Parsing** is the process of getting tokens from the lexical analyser and obtains a derivation for the sequence of tokens and builds a parse tree.
- Thus, if the program is synthetically correct, the parse tree is generated.
- If a derivation for the sequence of tokens does not exist, i.e., if the program is synthetically wrong, it results in syntax error and the parser displays the appropriate error messages.
- The **parse tree** is also called as **syntax tree**.
- **Parser** also called **syntax analyser** this is the one which does parsing.

# The role of parser



There are 3 general types of parsers for grammars:

1. Universal,
2. Top-down, and
3. Bottom-up.

- **Universal parsing** methods such as the Cocke-Younger-Kasami algorithm and Earley's algorithm can parse any grammar. These methods are inefficient.
- The methods commonly used in compilers can be classified as being either **top-down** or **bottom-up**.
- **Top-down method** builds parse trees from the top(root) to the bottom(leaves), while **bottom-up** method starts from the leaves and works their way up to the root.
- In either case, the input to the parser is scanned from left to right, one symbol at a time.

# Representative Grammar:

- Constructs that begin with keywords like while or int, are relatively easy to parse, because the keyword guides the choice of the grammar production that must be applied to match the input.
- Associativity and precedence are captured in the following grammar:

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow (E) \mid \mathbf{id} \end{aligned}$$

- ‘E’ represents ‘**expressions**’ consisting of ‘**terms**’ separated by ‘+’ signs, ‘T’ represents ‘**terms**’ consisting of ‘**factors**’ separated by ‘\*’ signs, and ‘F’ represents ‘**factors**’ that can be either ‘**parenthesized expressions**’ or ‘**identifiers**’.

# Uses of grammars

$$E \rightarrow E + T \mid T$$
$$T \rightarrow T * F \mid F$$
$$F \rightarrow (E) \mid \mathbf{id}$$
$$E \rightarrow TE'$$
$$E' \rightarrow +TE' \mid \varepsilon$$
$$T \rightarrow FT'$$
$$T' \rightarrow *FT' \mid \varepsilon$$
$$F \rightarrow (E) \mid \mathbf{id}$$

# Syntax Error Handling:

- Common programming errors:

1. **Lexical errors** – misspellings of identifiers
2. **Synthetic errors** – misplaced of semicolons or braces
3. **Semantic errors** – type mismatches between operators and operands
4. **Logical errors** – anything from incorrect reasoning on the part of program.

- Error handler goals:

1. Report the presence of errors clearly and accurately.
2. Recover from each error quickly enough to detect subsequent errors.
3. Add minimal overhead to the processing of correct programs.

# Error-recover strategies

- Panic mode recovery
  - Discard input symbol one at a time until one of designated set of synchronization tokens is found
- Phrase level recovery
  - Replacing a prefix of remaining input by some string that allows the parser to continue
- Error productions
  - Augment the grammar with productions that generate the erroneous constructs
- Global correction
  - Choosing minimal sequence of changes to obtain a globally least-cost correction

# Context Free Grammars:

- **Definition:** The context free grammar in short a CFG is 4 tuple,

$G=(V, T, P, S)$  where,

- **V** – is set of variables. The variables are also called **non-terminals**.
- **T** – is set of **terminals**.
- **P** – is set of **productions**. All productions in P are of the form  $A \rightarrow \alpha$  where A is a non-terminal and  $\alpha$  is string of grammar symbols.
- **S** – is the **start symbol**.
  
- E.g.:  $\text{stmt} \rightarrow \text{if (expr) stmt else stmt}$

1. **Terminals:** are the basic symbols from which strings are formed. The term “token name” is a synonym for ‘terminal’ and frequently we will use the word token for terminal when it is clear that we are talking about just token name.
  - E.g.: the terminals are keywords ‘if’ and ‘else’ and symbols ‘(’ and ‘)’.
2. **Non-terminals:** are synthetic variables that denote sets of strings. The set of strings denoted by non terminals helps to define the language generated by the grammar. Non terminals impose a hierarchical structure on the language that is key to syntax analysis and translation.
  - E.g.: ‘stmt’ and ‘expr’ are non-terminals

3. **Start Symbol:** In a grammar, one non terminal is distinguished as the start symbol, and the set of strings it denotes is the language generated by the grammar.
4. **Production:** of a grammar specify the manner in which the terminals and non terminals can be combined to form strings. Each production consists of:
  - a. A non terminal called the **head or left side** of the production; this production defines some of the strings denoted by the head.
  - b. The symbol  $\rightarrow$  sometimes  $::=$  has been used in place of arrow.
  - c. A **body or right side** consisting of zero or more terminals and non terminals. The components of the body describe one way in which strings of the non terminal at the head can be constructed.

# Notational Conventions:

1. These symbols are **terminals**:
  - (a) Lowercase letters early in the alphabet, such as a, b, c, d, etc.
  - (b) Operator symbols such as +, \*, -, /, and so on.
  - (c) Punctuation symbols such as parentheses, comma, and so on.
  - (d) The digits 0, 1 . . . 9.
  - (e) Boldface strings such as 'id' or 'if', each of which represents a single terminal symbol.

## 2. These symbols are **non terminals**:

- (a) Uppercase letters in the alphabet, such as A, B, C.
  - (b) The letter S, which, when it appears, is usually the start symbol.
  - (c) Lowercase, italic names such as *expr* or *stmt*.
  - (d) When discussing programming constructs, uppercase letters may be used to represent Non terminals for the constructs. For example, non terminal for expressions, terms, and factors are often represented by E, T, and F, respectively.
- 
- 3. Uppercase letters in the alphabet, such as, X, Y, Z, represent grammar symbols; i.e., either non-terminals or terminals.
  - 4. Lowercase letters in the alphabet, u, v, ..., z, represents (possibly empty) strings of terminals

5. Lowercase Greek letters,  $\alpha$ ,  $\beta$ ,  $\gamma$ , for example, represent strings of grammar symbols. Thus, a generic production can be written as,  $A \rightarrow \alpha$ , where  $A$  is the head and  $\alpha$  is the body.
6. A set of productions,  $A \rightarrow \alpha_1$ ,  $A \rightarrow \alpha_2$ , ...  $A \rightarrow \alpha_k$  with a common head  $A$ , may be written as  $A \rightarrow \alpha_1 \mid \alpha_2 \mid \dots \mid \alpha_k$ .
7. Unless stated otherwise, the head of the first production is the start symbol.

# Context free grammars

- Terminals
- Nonterminals
- Start symbol
- productions

expression  $\rightarrow$  expression + term

expression  $\rightarrow$  expression – term

expression  $\rightarrow$  term

term  $\rightarrow$  term \* factor

term  $\rightarrow$  term / factor

term  $\rightarrow$  factor

factor  $\rightarrow$  (expression)

factor  $\rightarrow$  **id**

# Derivations:

- **Definition:** The process of obtaining string of terminals and/or non-terminals from the start symbol by applying some set of production (or all productions) is called **derivation**.
- E.g.: if  $A \rightarrow \alpha B \gamma$  and  $B \rightarrow \beta$  are the productions, the string  $\alpha \beta \gamma$  can be obtained from A-production as shown,

$$A \Rightarrow \alpha B \gamma$$

$$A \Rightarrow \alpha \beta \gamma$$

- the above derivation can also be written as:
- $A \xRightarrow{+} \alpha \beta \gamma$

# Derivations

- Productions are treated as rewriting rules to generate a string
- Rightmost and leftmost derivations
  - $E \rightarrow E + E \mid E * E \mid -E \mid (E) \mid \mathbf{id}$
  - Derivations for  $\mathbf{-(id+id)}$ 
    - $E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E+E) \Rightarrow -(\mathbf{id}+E) \Rightarrow -(\mathbf{id}+\mathbf{id})$

•The two types of derivations are:

**1. Leftmost derivation (LMD):**

- The process of obtaining a string of terminals from the sequence of replacements such that only leftmost non-terminal is replaced at each and every step is called leftmost derivation.

**2. Rightmost Derivation (RMD):**

- The process of obtaining a string of terminals from the sequence of replacements such that only rightmost non-terminal is replaced at each and every step is called rightmost derivation.
- Example:  $E \rightarrow E + E \mid E * E \mid (E) \mid id$   
Obtain string:  $id + id * id$

# Sentential Form of G (Sentence of G):

- If  $S \xRightarrow{*} \alpha$ , where  $S$  is the start symbol of the grammar  $G$ , we say that  $\alpha$  is a sentential form of  $G$ .
- The sentential form may contain both terminals and non-terminals and may be empty.
- A sentence of  $G$  is a sentential form with no non-terminals.
- The language generated by a grammar is its set of sentences. Thus, a string of terminals  $w$  is in  $L(G)$ , the language generated by  $G$ , if and only if  $w$  is a sentences of  $G$  (or  $\overset{*}{S} \Rightarrow w$ ).
- A language that can be generated by a grammar is context-free grammar.

- **Type of Sentential Form:**

- 1. Left Sentential Form:**

- If there is a derivation of the form  $S \xRightarrow{*} \alpha$ , where at each step in the derivation process only a leftmost variable is replaced, then  $\alpha$  is called left-sentential form of  $G$ .

- 2. Right Sentential Form:**

- If there is a derivation of the form  $S \xRightarrow{*} \alpha$ , where at each step in the derivation process only a rightmost variable is replaced, then  $\alpha$  is called right-sentential form of  $G$ .
- Example:  $E \rightarrow E + E \mid E * E \mid (E) \mid id$
- Derive the leftmost derivation for the above grammar.

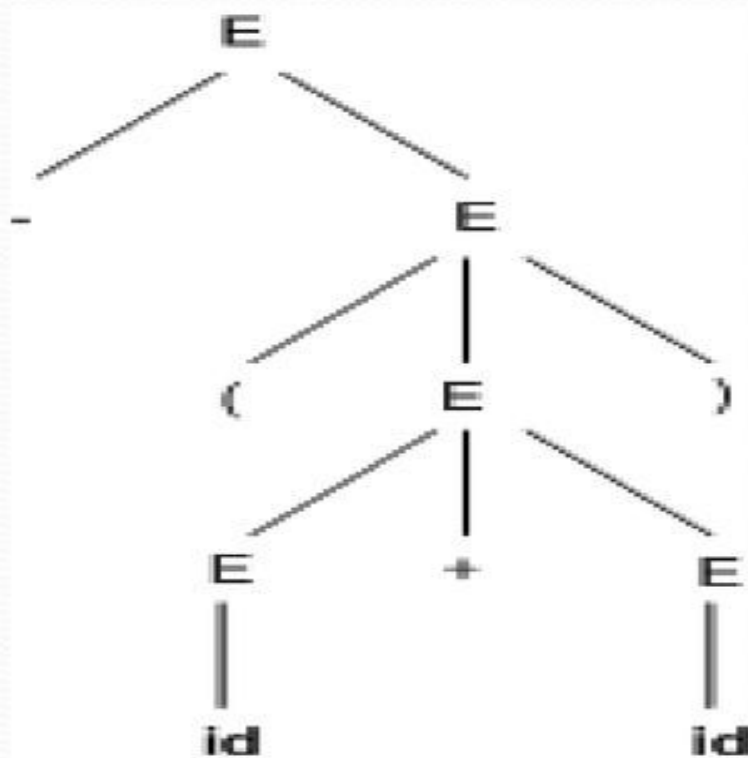
{The string of grammar symbols obtained in each step of leftmost derivation are the various left-sentential form of  $G$ .}

# Parse Trees and Derivations:

- A **parse tree** is a graphical representation of a derivation that filters out the order in which productions are applied to replace non-terminals.
- Each interior node of a parse tree represents the application of the production.
- The interior node is labeled with a non-terminal 'A' in the head of the production; the children of the node are labeled from left to right, by the symbols in the body of the production by which this 'A' was replaced during the derivation.
- **Example: Construct a parse tree for the given grammar:  $E \rightarrow E + E \mid E * E \mid (E) \mid id$**
- The leaves of the parse tree are labelled by non terminals or terminals, and read from left to right, constitute a sentential form, called the yield or frontier of the tree.
- Read from left to right:  $id + id * id$  this is the yield of the given parse tree

# Parse trees

- $-(\text{id}+\text{id})$
- $E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E+E) \Rightarrow -(\text{id}+E) \Rightarrow -(\text{id}+\text{id})$

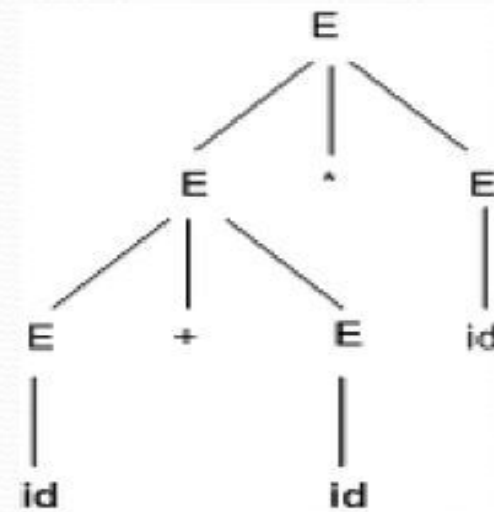
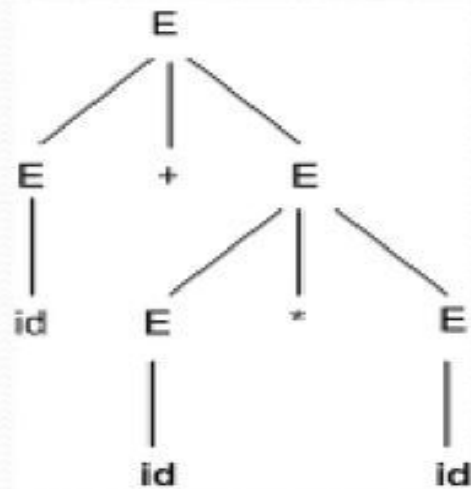


# Ambiguity:

- A grammar that produces more than one parse tree for some sentence is said to be ambiguous.
- An ambiguous grammar is the one that produces more than one leftmost derivation or more than one right most derivation for the same sentence.
- Example: the arithmetic expression grammar below permits 2 distinct leftmost derivation for this sentence:  $\text{id} + \text{id} * \text{id}$ .
- Grammar:  $E \rightarrow E + E \mid E * E \mid (E) \mid \text{id}$
- Since the 2 parse trees are different for the same sentence  $\text{id} + \text{id} * \text{id}$  by applying leftmost derivation, the grammar is ambiguous.
- It is convenient to use carefully chosen ambiguous grammars, together with **disambiguating rules** that 'throw away' undesirable parse trees, leaving only one tree for each sentence

# Ambiguity

- For some strings there exist more than one parse tree
- Or more than one leftmost derivation
- Or more than one rightmost derivation
- Example:  $\text{id} + \text{id} * \text{id}$



# Writing a Grammar:

- Grammars are capable of describing most, but not all, of the syntax of programming languages.
- For instance, the requirement that identifiers be declared before they are used, cannot be described by a context free grammar.
- Therefore, the sequences of tokens accepted by a parser form a superset of the programming language; subsequent phases of the compiler must analyse the output of the parser to ensure compliance with rules that are not checked by the parser.

# Lexical vs. Syntactic Analysis:

- To use regular expressions to define the lexical syntax of a language, there are several reasons.
1. Separating the syntactic structure of a language into lexical and non lexical paths provides a convenient way of modularization the front end of a compiler into 2 manageable sized components.
  2. The lexical rules of a language are frequently quite simple, hi to describe them we do not need a notation as powerful as grammars.
  3. Regular expressions generally provide a more concise hand easier to understand for tokens than grammars.
  4. More efficient lexical analysers can be constructed automatically from regular expression than from arbitrary grammar.

- Regular expressions are most useful for describing the structure of constructs such as identifiers, constants, keywords, and white space.
- Grammars are most useful for describing nested structures such as balanced parenthesis, matching begin-end's, corresponding if-then-else's, and so on.
- These nested structures cannot be described by regular expressions.

# Eliminating Ambiguity:

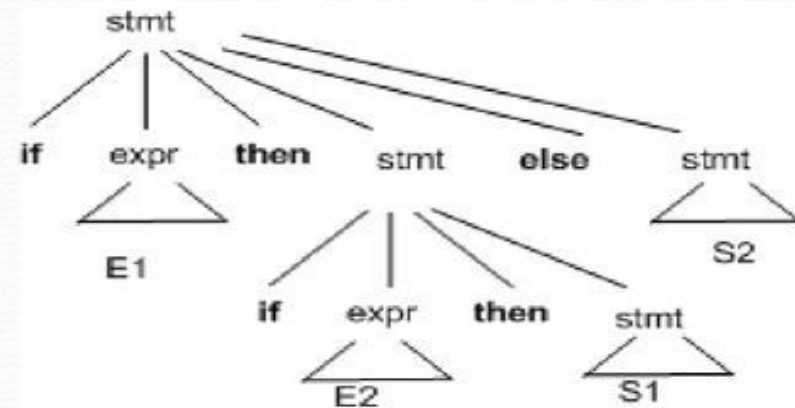
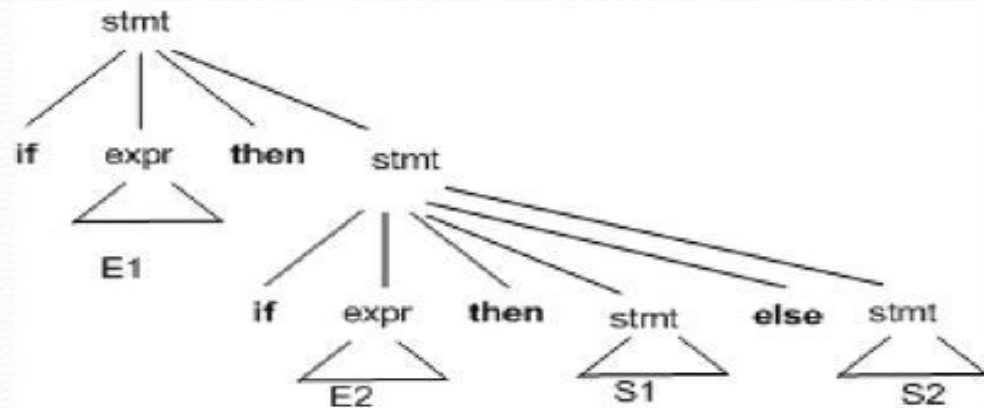
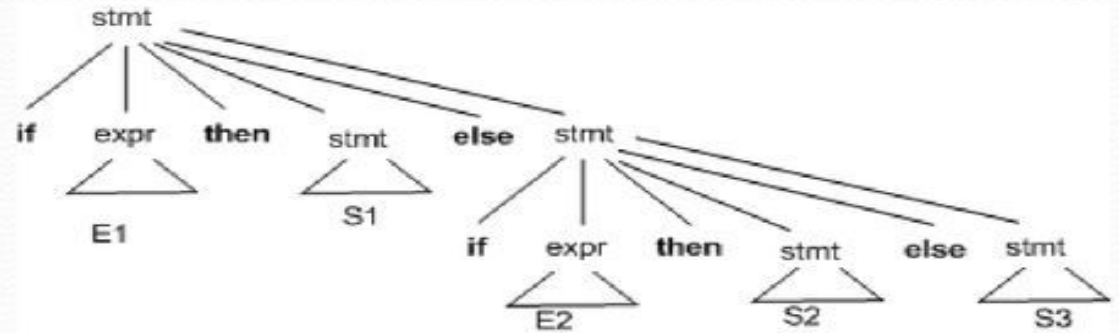
- Sometimes an ambiguous grammar can be rewritten to eliminate the ambiguity.
- Example: eliminate the ambiguity from the following 'dangling-else' grammar:

**stmt -> if 'expr' then 'stmt'**  
**| If 'expr' then 'stmt' else 'stmt'**  
**| other**

- Here, other stands for any other statement.

# Elimination of ambiguity

stmt  $\rightarrow$  **if** expr **then** stmt  
| **if** expr **then** stmt **else** stmt  
| **other**



- According to the grammar, the compound conditional statement as the parse tree:

**if E1 then S1 else if E2 then S2 else S3**

- The grammar is ambiguous since the string below has 2 parse trees:

**if E1 then if E2 then S1 else S2**

- In all programming languages with conditional statements of this form, the first parse tree is preferred.
- The general rule is, ‘Match each else with the closest unmatched then’. This disambiguating rule can theoretically be incorporated directly into a grammar, but in practice it is rarely built into the productions.

# Elimination of left recursion

- A grammar is left recursive if it has a non-terminal  $A$  such that there is a derivation  $A \xRightarrow{\pm} A \alpha$
- Top down parsing methods cant handle left-recursive grammars
- A simple rule for direct left recursion elimination:
  - For a rule like:
    - $A \rightarrow A \alpha \mid \beta$
  - We may replace it with
    - $A \rightarrow \beta A'$
    - $A' \rightarrow \alpha A' \mid \epsilon$

- Example: consider the expression grammar:

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow (E) \mid \mathbf{id} \end{aligned}$$

- The non-left recursive expression grammar:

$$\begin{aligned} E &\rightarrow TE' \\ E' &\rightarrow +TE' \mid \epsilon \\ T &\rightarrow FT' \\ T' &\rightarrow *FT' \mid \epsilon \\ F &\rightarrow (E) \mid \mathbf{id} \end{aligned}$$

is obtained by eliminating immediate left recursion from the expression grammar.

- The left recursive pair of productions  $E \rightarrow E+T \mid T$  are replaced by  $E \rightarrow TE'$  and  $E' \rightarrow +TE' \mid \epsilon$

- Immediate left recursion can be eliminated by the following technique, which works for any number of A-productions.

- First, group the productions as:

$$A \rightarrow A\alpha_1 \mid A\alpha_2 \mid A\alpha_3 \mid \dots \mid A\alpha_m \mid \beta_1 \mid \beta_2 \mid \beta_3 \mid \dots \mid \beta_m$$

where no  $\beta_i$  begins with an A.

- Then, replace the A-production by,

$$A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \dots \mid \beta_n A'$$

$$A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \dots \mid \alpha_m A' \mid \epsilon$$

- The non-terminal A generates the C strings as before but is no longer left recursive.
- This procedure eliminates all left recursion from the A and A' productions, but it does not eliminate left recursion involving derivations of 2 or more steps.

# Left recursion elimination (cont.)

- There are cases like following
  - $S \rightarrow Aa \mid b$
  - $A \rightarrow Ac \mid Sd \mid \varepsilon$
- Left recursion elimination algorithm:
  - Arrange the nonterminals in some order  $A_1, A_2, \dots, A_n$ .
  - For (each  $i$  from 1 to  $n$ ) {
    - For (each  $j$  from 1 to  $i-1$ ) {
      - Replace each production of the form  $A_i \rightarrow A_j \gamma$  by the production  $A_i \rightarrow \delta_1 \gamma \mid \delta_2 \gamma \mid \dots \mid \delta_k \gamma$  where  $A_j \rightarrow \delta_1 \mid \delta_2 \mid \dots \mid \delta_k$  are all current  $A_j$  productions
      - }
      - Eliminate left recursion among the  $A_i$ -productions
    - }

# Left factoring

- Left factoring is a grammar transformation that is useful for producing a grammar suitable for predictive or top-down parsing.
- Consider following grammar:
  - Stmt  $\rightarrow$  **if** expr **then** stmt **else** stmt
  - | **if** expr **then** stmt
- On seeing input **if** it is not clear for the parser which production to use
- We can easily perform left factoring:
  - If we have  $A \rightarrow \alpha \beta_1 \mid \alpha \beta_2$  then we replace it with
    - $A \rightarrow \alpha A'$
    - $A' \rightarrow \beta_1 \mid \beta_2$

# Left factoring (cont.)

- Algorithm
  - For each non-terminal  $A$ , find the longest prefix  $\alpha$  common to two or more of its alternatives. If  $\alpha \neq \epsilon$ , then replace all of  $A$ -productions  $A \rightarrow \alpha \beta_1 \mid \alpha \beta_2 \mid \dots \mid \alpha \beta_n \mid \gamma$  by
    - $A \rightarrow \alpha A' \mid \gamma$
    - $A' \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$
- Example:
  - $S \rightarrow I E t S \mid i E t S e S \mid a$
  - $E \rightarrow b$

# Top Down Parsing

# Introduction

- A Top-down parser tries to create a parse tree from the root towards the leafs scanning input from left to right
- It can be also viewed as finding a leftmost derivation for an input string
- Example:  $\text{id} + \text{id} * \text{id}$

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \epsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \epsilon$

$F \rightarrow (E) \mid \mathbf{id}$

# Recursive descent parsing

- Consists of a set of procedures, one for each nonterminal
- Execution begins with the procedure for start symbol
- A typical procedure for a non-terminal

```
void A() {  
    choose an A-production,  $A \rightarrow X_1 X_2 \dots X_k$   
    for (i=1 to k) {  
        if ( $X_i$  is a nonterminal  
            call procedure  $X_i()$ ;  
        else if ( $X_i$  equals the current input symbol a)  
            advance the input to the next symbol;  
        else /* an error has occurred */  
    }  
}
```

# Recursive descent parsing (cont)

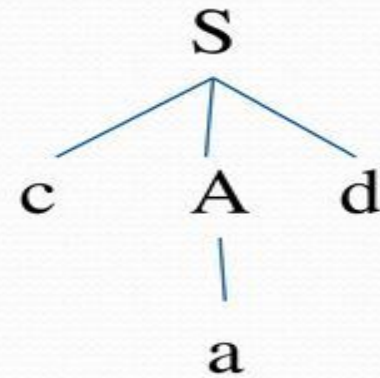
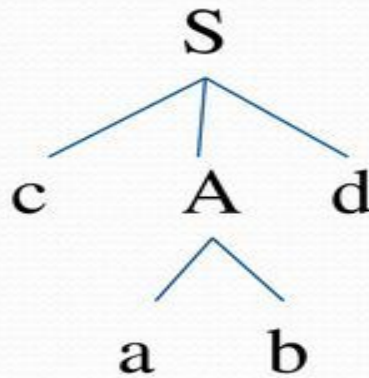
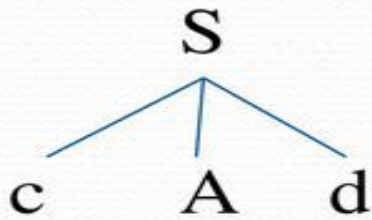
- General recursive descent may require backtracking
- The previous code needs to be modified to allow backtracking
- In general form it can't choose an A-production easily.
- So we need to try all alternatives
- If one failed the input pointer needs to be reset and another alternative should be tried
- Recursive descent parsers can't be used for left-recursive grammars

# Example

$S \rightarrow cAd$

$A \rightarrow ab \mid a$

Input: cad



# First and Follow

- $\text{First}()$  is set of terminals that begins strings derived from
- If  $\alpha \xRightarrow{*} \varepsilon$  then  $\varepsilon$  is also in  $\text{First}(\varepsilon)$
- In predictive parsing when we have  $A \rightarrow \alpha \mid \beta$ , if  $\text{First}(\alpha)$  and  $\text{First}(\beta)$  are disjoint sets then we can select appropriate  $A$ -production by looking at the next input
- $\text{Follow}(A)$ , for any nonterminal  $A$ , is set of terminals  $a$  that can appear immediately after  $A$  in some sentential form
  - If we have  $S \xRightarrow{*} \alpha A a \beta$  for some  $\alpha$  and  $\beta$  then  $a$  is in  $\text{Follow}(A)$
- If  $A$  can be the rightmost symbol in some sentential form, then  $\$$  is in  $\text{Follow}(A)$

# Computing First

- To compute  $\text{First}(X)$  for all grammar symbols  $X$ , apply following<sup>\*</sup> rules until no more terminals or  $\epsilon$  can be added to any First set:
  1. If  $X$  is a terminal then  $\text{First}(X) = \{X\}$ .
  2. If  $X$  is a nonterminal and  $X \rightarrow Y_1 Y_2 \dots Y_k$  is a production for some  $k \geq 1$ , then place  $a$  in  $\text{First}(X)$  if for some  $i$   $a$  is in  $\text{First}(Y_i)$  and  $\epsilon$  is in all of  $\text{First}(Y_1), \dots, \text{First}(Y_{i-1})$  that is  $Y_1 \dots Y_{i-1} \xRightarrow{*} \epsilon$ . if  $\epsilon$  is in  $\text{First}(Y_j)$  for  $j=1, \dots, k$  then add  $\epsilon$  to  $\text{First}(X)$ .
  3. If  $X \rightarrow \epsilon$  is a production then add  $\epsilon$  to  $\text{First}(X)$
- Example!

# Computing follow

- To compute  $\text{FOLLOW}(A)$  for all nonterminals  $A$ , apply following rules until nothing can be added to any follow set:
  1. Place  $\$$  in  $\text{Follow}(S)$  where  $S$  is the start symbol
  2. If there is a production  $A \rightarrow \alpha B \beta$  then everything in  $\text{First}(\beta)$  except  $\epsilon$  is in  $\text{Follow}(B)$ .
  3. If there is a production  $A \rightarrow B$  or a production  $A \rightarrow \alpha B \beta$  where  $\text{First}(\beta)$  contains  $\epsilon$ , then everything in  $\text{Follow}(A)$  is in  $\text{Follow}(B)$
- Example!

- **To Compute FOLLOW()**

- **FOLLOW( $X$ )** is the set of terminals that can appear immediately to the right of  $X$  in some derivation.

- **Rules for FOLLOW()**

1. **FOLLOW( $S$ )** always contains  $\$$  (end of input).
2. If a non-terminal  $A$  is followed by a terminal  $b$  in a production  $A \rightarrow \alpha b \beta$ , then  $b \in \text{FOLLOW}(A)$ .
3. If a non-terminal  $A$  is followed by a non-terminal  $B$ , then  $\text{FIRST}(B) \subseteq \text{FOLLOW}(A)$  (except  $\epsilon$ ).
4. If  $\epsilon \in \text{FIRST}(B)$ , then  $\text{FOLLOW}(A) \subseteq \text{FOLLOW}(B)$ .

# LL(1) Grammars

- Predictive parsers are those recursive descent parsers needing no backtracking
- Grammars for which we can create predictive parsers are called LL(1)
  - The first L means scanning input from left to right
  - The second L means leftmost derivation
  - And 1 stands for using one input symbol for lookahead
- A grammar  $G$  is LL(1) if and only if whenever  $A \rightarrow \alpha \mid \beta$  are two distinct productions of  $G$ , the following conditions hold:
  - For no terminal  $a$  do  $\alpha$  and  $\beta$  both derive strings beginning with  $a$
  - At most one of  $\alpha$  or  $\beta$  can derive empty string<sub>\*</sub>
  - If  $\alpha \Rightarrow^* \varepsilon$  then  $\beta$  does not derive any string beginning with a terminal in  $\text{Follow}(A)$ .

# Construction of predictive parsing table

- For each production  $A \rightarrow \alpha$  in grammar do the following:
  1. For each terminal  $a$  in  $\text{First}(\alpha)$  add  $A \rightarrow$  in  $M[A,a]$
  2. If  $\epsilon$  is in  $\text{First}(\alpha)$ , then for each terminal  $b$  in  $\text{Follow}(A)$  add  $A \rightarrow \epsilon$  to  $M[A,b]$ . If  $\epsilon$  is in  $\text{First}(\alpha)$  and  $\$$  is in  $\text{Follow}(A)$ , add  $A \rightarrow \epsilon$  to  $M[A,\$]$  as well
- If after performing the above, there is no production in  $M[A,a]$  then set  $M[A,a]$  to error

# Example

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \epsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \epsilon$

$F \rightarrow (E) \mid \mathbf{id}$

|    | First            | Follow        |
|----|------------------|---------------|
| F  | {(,id}           | {+, *, ), \$} |
| T  | {(,id}           | {+, ), \$}    |
| E  | {(,id}           | {), \$}       |
| E' | {+, $\epsilon$ } | {), \$}       |
| T' | {*, $\epsilon$ } | {+, ), \$}    |

| Non - terminal | Input Symbol                |                           |                       |                     |                           |                           |
|----------------|-----------------------------|---------------------------|-----------------------|---------------------|---------------------------|---------------------------|
|                | id                          | +                         | *                     | (                   | )                         | \$                        |
| E              | $E \rightarrow TE'$         |                           |                       | $E \rightarrow TE'$ |                           |                           |
| E'             |                             | $E' \rightarrow +TE'$     |                       |                     | $E' \rightarrow \epsilon$ | $E' \rightarrow \epsilon$ |
| T              | $T \rightarrow FT'$         |                           |                       | $T \rightarrow FT'$ |                           |                           |
| T'             |                             | $T' \rightarrow \epsilon$ | $T' \rightarrow *FT'$ |                     | $T' \rightarrow \epsilon$ | $T' \rightarrow \epsilon$ |
| F              | $F \rightarrow \mathbf{id}$ |                           |                       | $F \rightarrow (E)$ |                           |                           |

# Another example

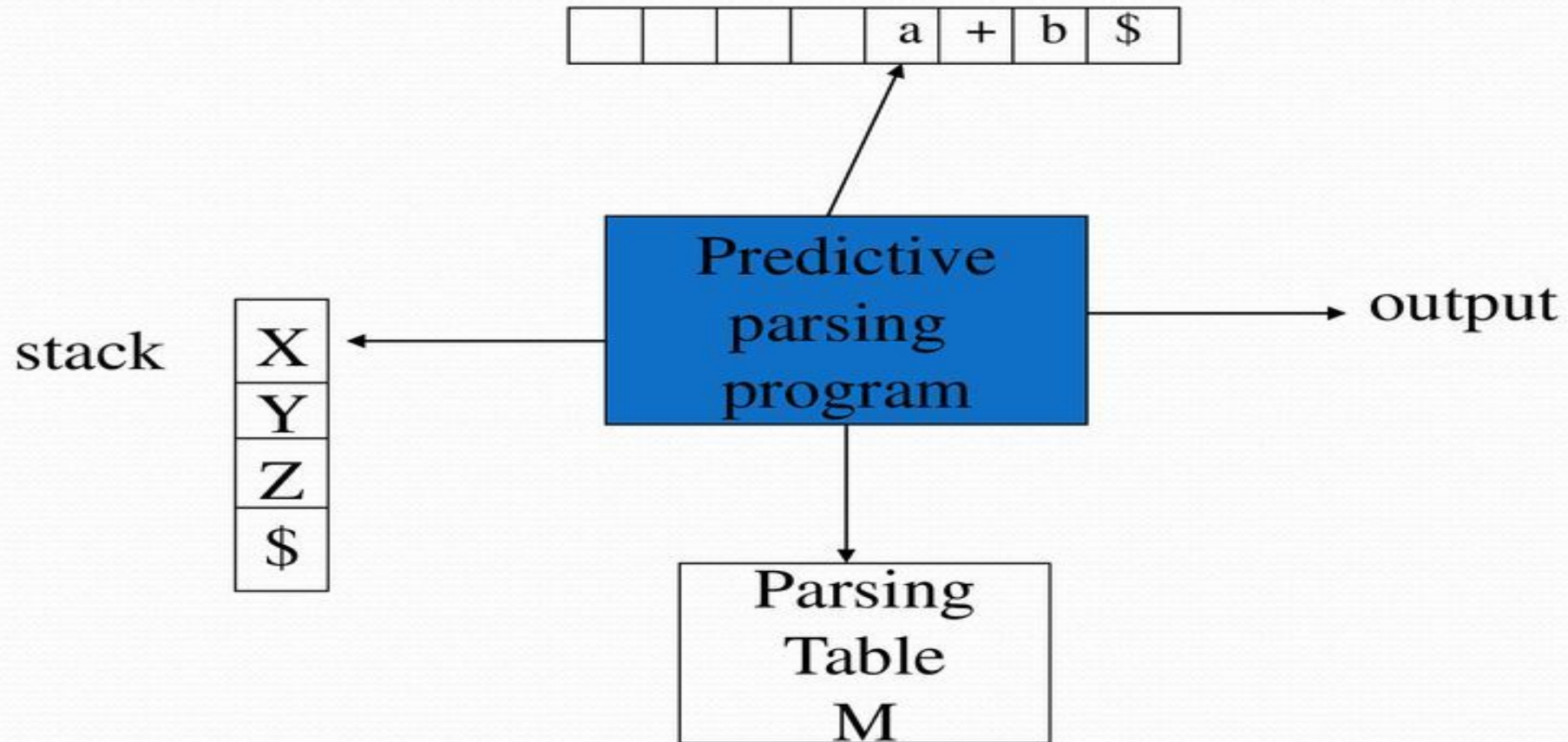
$S \rightarrow iEtSS' \mid a$

$S' \rightarrow eS \mid \epsilon$

$E \rightarrow b$

| Non -<br>terminal | Input Symbol      |                   |                                                  |                        |   |                           |
|-------------------|-------------------|-------------------|--------------------------------------------------|------------------------|---|---------------------------|
|                   | a                 | b                 | e                                                | i                      | t | \$                        |
| S                 | $S \rightarrow a$ |                   |                                                  | $S \rightarrow iEtSS'$ |   |                           |
| S'                |                   |                   | $S' \rightarrow \epsilon$<br>$S' \rightarrow eS$ |                        |   | $S' \rightarrow \epsilon$ |
| E                 |                   | $E \rightarrow b$ |                                                  |                        |   |                           |

# Non-recursive predicting parsing



# Predictive parsing algorithm

```
Set ip point to the first symbol of w;  
Set X to the top stack symbol;  
While (X<>$) { /* stack is not empty */  
    if (X is a) pop the stack and advance ip;  
    else if (X is a terminal) error();  
    else if (M[X,a] is an error entry) error();  
    else if (M[X,a] = X->Y1Y2..Yk) {  
        output the production X->Y1Y2..Yk;  
        pop the stack;  
        push Yk,...,Y2,Y1 on to the stack with Y1 on top;  
    }  
    set X to the top stack symbol;  
}
```

# Example

- $\text{id+id*id\$}$

| Matched | Stack | Input               | Action |
|---------|-------|---------------------|--------|
|         | E\$   | $\text{id+id*id\$}$ |        |

# Error Recovery in Predictive Parsing

1. Panic Mode
2. Phrase-level Recovery

[For explanation refer notes]



# Bottom-up Parsing

# Introduction

- Constructs parse tree for an input string beginning at the leaves (the bottom) and working towards the root (the top)
- Example:  $\text{id} * \text{id}$

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow (E) \mid \mathbf{id} \end{aligned}$$

# Reductions:

- During bottom up parsing, the given string of terminals are reduced to the start symbol.
- The process of searching for right hand side of production in a string consisting of terminals and non terminals and replacing it with corresponding left hand side of the production is called **reduction**.
- The right most derivation to get the string  $id * id$  is:

$$E \Rightarrow T$$

$$\Rightarrow T * F$$

$$\Rightarrow T * id$$

$$\Rightarrow F * id$$

$$\Rightarrow id * id$$

# Handle pruning

- A Handle is a substring that matches the body of a production and whose reduction represents one step along the reverse of a rightmost derivation

| Right sentential form | Handle | Reducing production |
|-----------------------|--------|---------------------|
| id*id                 | id     | $F \rightarrow id$  |
| F*id                  | F      | $T \rightarrow F$   |
| T*id                  | id     | $F \rightarrow id$  |
| T*F                   | T*F    | $E \rightarrow T*F$ |

# Shift-reduce parser

- The general idea is to shift some symbols of input to the stack until a reduction can be applied
- At each reduction step, a specific substring matching the body of a production is replaced by the nonterminal at the head of the production
- The key decisions during bottom-up parsing are about when to reduce and about what production to apply
- A reduction is a reverse of a step in a derivation
- The goal of a bottom-up parser is to construct a derivation in reverse:
  - $E \Rightarrow T \Rightarrow T * F \Rightarrow T * id \Rightarrow F * id \Rightarrow id * id$

# Shift reduce parsing

- A stack is used to hold grammar symbols
- Handle always appear on top of the stack
- Initial configuration:

| Stack | Input |
|-------|-------|
| \$    | w\$   |

- Acceptance configuration

| Stack | Input |
|-------|-------|
| \$S   | \$    |

# Shift-Reduce Parsing

- The primary operations are shift and reduce, there are actually four possible actions a shift-reduce parser can make:

(1) shift, (2) reduce, (3) accept, and (4) error.

1. **Shift:** Shift the next input symbol onto the top of the stack.
2. **Reduce:** The right end of the string to be reduced must be at the top of the stack. Locate the left end of the string within the stack and decide with what nonterminal to replace the string.
3. **Accept:** Announce successful completion of parsing.
4. **Error:** Discover a syntax error and call an error recovery routine

# Conflicts During Shift-Reduce Parsing

- There are two types of conflicts.

1. Shift/reduce conflicts

2. Reduce/Reduce conflicts

- **Shift-Reduce Conflict:** during parsing, by knowing the content of the stack and the next input symbol, if the parser cannot decide whether to shift the input symbol on to the stack or to reduce the angle on top of the stack, it results in **shift-reduce conflict**.

- **Reduce-Reduce Conflict:** during parsing, if shift reduce parser finds a handle on the top of the stack, the parser has to reduce the handle to the left hand side of the production. But, if 2 or more productions have same handle on right hand side of the production, the parser cannot decide which production has to be selected for reducing. This conflict is called **reduce-reduce conflict**.